

自主學習成果
投籃動作檢測系統
成功高中 郭昶佑

（一）研究背景與目標

我從小就喜歡打籃球，國小和國中也加入了球隊，而我這種身材體能條件較不出色，投籃是我最大利器。當時我自認是個射手，然而當退到三分線後時，我卻發現自己使不上力，突然間好像不會打球了，因此我想寫出一個程式來修正自己的投籃動作，讓我能投的到三分球！

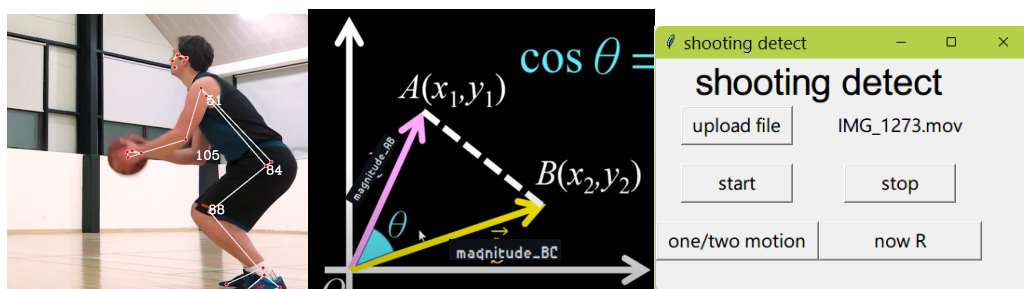
（二）研究方法與過程

在我看了很多介紹投籃的影片後，各式各樣的名詞，像屈膝屈髖、手快腳慢、一段式二段式等等，我發現他們都能靠分析投籃時的關節角度來將投籃動作給量化，如此我們就可以將其會製成圖表，清楚分析動作的異同。

第一步：選擇要使用mediapipe的PoseDrager節點，我選擇的夾角分別時手臂、肩膀、髖、膝蓋四個。

第二步：運用內積來算出三點所連出向量的夾角，標示在OpenCV的畫面上及用matplotlib記錄成圖表。

第三步：用tkinter做UI，上面包含選擇要分析的影片、開始/暫停按鈕。



圖(一) PoseDrager節點 圖(二) 內積運算示意圖 圖(三) 使用者UI

第四步：減少誤差，把影片拿去分析後發現不同拍攝角度也會造成結果上的誤差，於是我新增了選擇是要偵測人左側節點還是右側節點的功能。

製作過程中我又想到能新增的功能，於是就有了

第五步：準備訓練資料，從網路上篩選適合的球員投籃影片，並把背景用色塊擋起來，不讓後面的人影響偵測球員，最後一段式、二段式各蒐集了三個球員，一個球員蒐集兩個角度各5次影片，總共 $2*3*5*2=60$ 個資料。

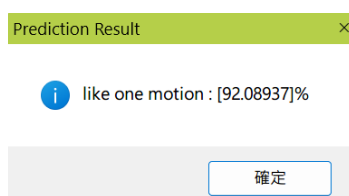


id	Motion	Ballbow_list	Rahoulder_list	Rbody_list	Rknee_list
14	one	[142, 158, 150, 1]	[6, 3, 4, 8, 4, 4, 3]	[152, 141, 139, 1]	[123, 95, 95, 96]
15	one	[138, 142, 144, 1]	[40, 6, 8, 10, 13]	[173, 131, 129, 1]	[169, 148, 150, 1]
16	one	[97, 93, 78, 66, 6]	[1, 1, 5, 10, 9, 10]	[169, 153, 149, 1]	[151, 118, 124, 1]
17	one	[3, 22, 39, 37, 22]	[37, 32, 34, 33, 3]	[149, 146, 144, 1]	[142, 137, 135, 1]
18	one	[102, 96, 91, 87]	[20, 20, 26, 27, 4]	[144, 137, 128, 1]	[118, 112, 108, 1]
19	one	[38, 165, 178, 17]	[40, 29, 26, 22, 2]	[84, 129, 137, 13]	[85, 133, 141, 14]
20	one	[174, 176, 177, 1]	[1, 8, 14, 13, 16]	[130, 140, 139, 1]	[139, 153, 154, 1]
21	one	[122, 117, 125, 1]	[5, 7, 11, 15, 17]	[161, 159, 163, 1]	[170, 172, 178, 1]
22	one	[162, 133, 12, 16]	[139, 136, 33, 47]	[12, 8, 158, 165]	[83, 130, 163, 16]
23	one	[97, 107, 104, 13]	[49, 29, 21, 19, 2]	[160, 154, 151, 1]	[158, 152, 143, 1]
24	two	[52, 51, 73, 83, 9]	[53, 66, 44, 40, 3]	[175, 175, 164, 1]	[157, 161, 147, 1]
25	two	[10, 54, 150, 148]	[38, 44, 38, 35, 3]	[173, 175, 179, 1]	[159, 163, 162, 1]
26	two	[158, 163, 164, 1]	[10, 10, 10, 10, 5]	[158, 151, 149, 1]	[151, 143, 141, 1]
27	two	[168, 173, 153, 1]	[26, 27, 19, 17, 1]	[164, 170, 168, 1]	[171, 151, 146, 1]
28	two	[88, 87, 101, 104]	[4, 3, 4, 4, 6, 6, 4]	[170, 168, 163, 1]	[139, 138, 135, 1]
29	two	[96, 91, 90, 89, 8]	[37, 39, 40, 20, 1]	[168, 176, 171, 1]	[175, 158, 157, 1]
30	two	[111, 115, 125, 1]	[21, 13, 6, 5, 0, 1]	[173, 163, 152, 1]	[135, 135, 135, 1]
31	two	[83, 75, 71, 75, 7]	[6, 6, 0, 6, 7, 14]	[174, 173, 176, 1]	[162, 153, 153, 1]
32	two	[92, 91, 85, 84, 8]	[32, 44, 39, 36, 2]	[178, 173, 171, 1]	[170, 167, 161, 1]

圖(四) 用色塊阻擋雜訊

圖(五) 蒐集後的csv

第六步：訓練模型，因為每段訓練資料時間並不同，需要先把列表長度補0到相同長度為止，後面的處理就是基礎的tensorflow二元分類。



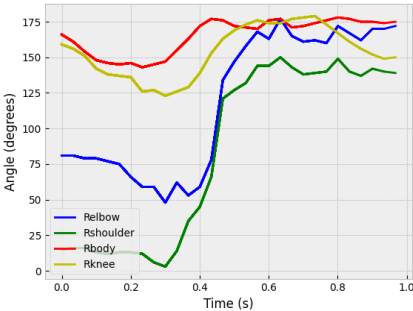
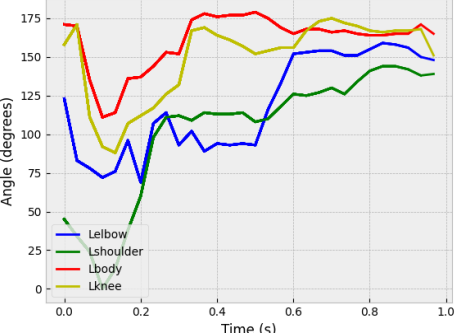
圖(六)判斷結果彈窗

(三) 研究結果與討論

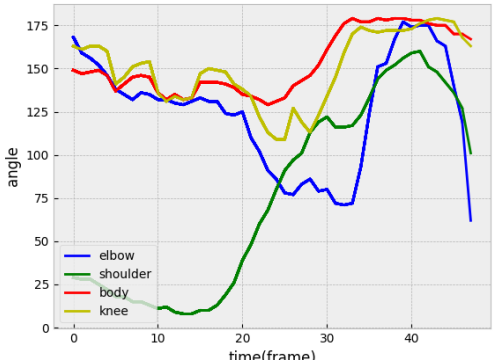
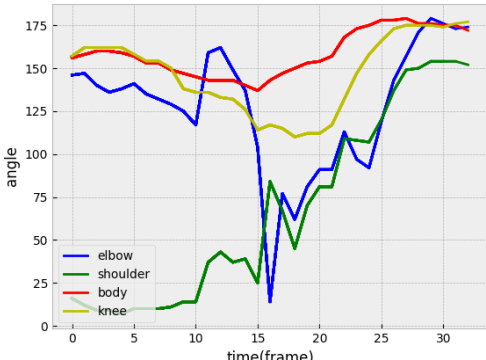
開始分析前先定義一下： 球投出時 =手肘（藍色）開始上升時，因為不管你的動作再怎麼奇怪，你都必須將你的小臂揮出，東西才可能丟的比較遠，你不妨試試看。

來介紹剛剛提到的一段式二段式，以及用圖表分析後得出的結論：

名稱	一段式(one motion)	二段式(two motion)
代表人物	Stephen Curry、Trae Young	Kobe Bryant、Ray Allen、林志傑

圖表	 圖(七) Stephen Curry 一段式投籃動作的關節角度變化	 圖(八) 林志傑 二段式投籃動作的關節角度變化
圖表分析	發力順暢、甩鞭效應、投的遠	先跳再投、0.3s到0.5s滯空、出手點高、不易被蓋火鍋

接著回到我原來目標投三分球，可以看到在調整前我的球投出時髖和膝已經打直，看起來跟林志傑一樣，但two motion需要靠著強大的核心和手臂力量來投籃，一般人如果想要更輕鬆投到三分可以往one motion的方式學，因為可以利用類似甩鞭效應來將力量從腳底往手最後在到球，讓力量在傳遞過程中有最小的損失。明瞭此道理且修改動作後，我的四條線能同時上升，也能有效運用到髖和膝蓋的力量。最終，我能輕鬆的投到了三分球了！

調正前	調整後
	

(四) 結論應用及展望

從結果我們可以知道one motion和two motion有各自的優勢，於是產生了像Dev in Booker 的混和型投手，在三分線外用one motion而在三分內則用two motion，放大兩種各自的優勢。我希望未來可以再新增

1. 自動辨識起跳、起球、投球階段的AI功能，分析時能更輕鬆判斷現在的動作大概對應到哪
2. 編輯影片開始結尾、放上色塊功能，而不用另外使用剪輯軟體。
3. 打包成exe，讓有興趣的人使用（可能電腦環境不乾淨，嘗試了很久都沒成功）

同時我現在也在做類似的專案，運用上述功能加上yolo追蹤籃球，除了分析關節角度，也分析球的起落速度的改變，一探絲滑運球秘密！

(五) 參考文獻

OpenCV Documentation. (n.d.). Retrieved from <https://opencv.org/>

MediaPipe Documentation. (n.d.). Retrieved from <https://developers.google.com/mediapipe/>

Matplotlib Documentation. (n.d.). Retrieved from <https://matplotlib.org/>

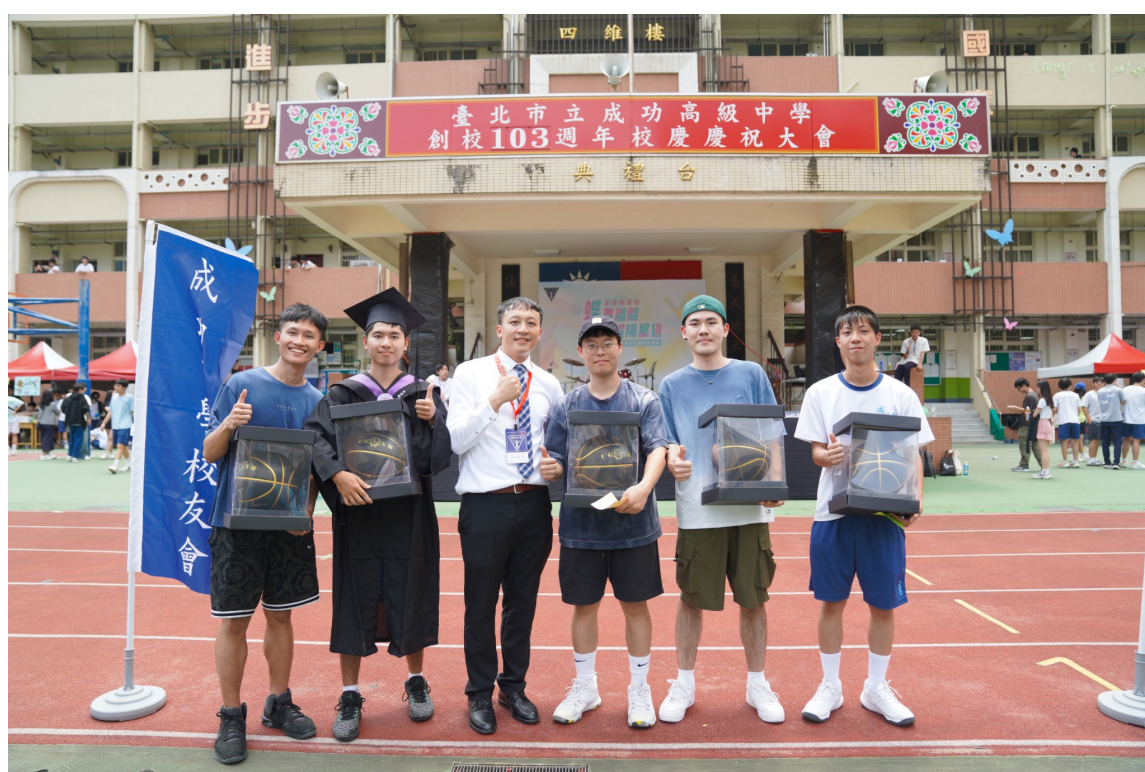
TensorFlow Documentation. (n.d.). Retrieved from <https://www.tensorflow.org/>

[YouTube]. (n.d.). AI辨識投籃訓練影片. Retrieved from <https://www.youtube.com/>

(六) 投稿sitcon 照片記錄



(七)投籃大賽得獎照片



(八) 海報

你會投籃嗎

透過動作分析，修正投籃姿勢

成功高中 郭昶佑

(一) 研究背景與目標

我自認是個射手，然而當退到三分線後時，卻發現自己使不上力，突然間好像不會投籃了，因此我想寫出一個程式來修正自己的投籃動作，讓我投得的到三分球！

(二) 研究方法與過程

我發現能靠分析投籃時的關節角度來將投籃動作量化，將其會製成圖表，分析動作的異同。

第一步：選擇mediapipe的PoseDrager節點，我選擇手臂、肩膀、髖、膝蓋四個。

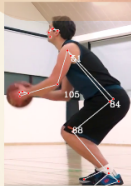
第二步：用內積來算出三點所連出向量的夾角，標示在OpenCV的畫面上及用matplotlib記錄成圖表。

第三步：用tkinter做UI，包含選擇要分析의影片、開始/暫停按鈕。

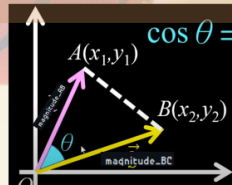
第四步：減少誤差，我發現不同拍攝角度也會造成結果上的誤差，於是新增選擇偵測人左/右側節點的功能。

第五步：準備訓練資料，從網路上篩選適合的球员投籃影片，並把背景用色塊擋起來控制雜訊，最後蒐集總共 $2*3*5*2=60$ 個資料。

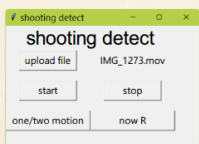
第六步：訓練模型，因為每段訓練資料時間並不同，需要先把列表長度補0到相同長度為止，後面的處理就是基礎的tensorflow二元分類。



圖(一) PoseDrager節點



圖(二) 內積運算示意圖



圖(三) 使用者UI



圖(四) 用色塊阻擋雜訊

id	Motion	Ballpose_list	Ballpose_list	Body_list	Body_list
14	one	[142, 150, 150, 1]	[6, 3, 4, 8, 4, 4, 3]	[158, 141, 139, 1]	[121, 95, 95, 96]
15	one	[138, 142, 144, 1]	[40, 6, 8, 10, 13]	[173, 131, 129, 1]	[149, 148, 150, 1]
16	one	[97, 93, 78, 66, 6]	[1, 1, 5, 10, 9, 10]	[149, 153, 149, 1]	[151, 118, 124, 1]
17	one	[3, 22, 39, 37, 22]	[37, 32, 34, 33, 3]	[149, 146, 144, 1]	[142, 137, 135, 1]
18	one	[102, 96, 91, 87]	[20, 20, 26, 22, 4]	[144, 137, 128, 1]	[116, 112, 108, 1]
19	one	[38, 165, 178, 17]	[40, 29, 26, 22, 2]	[184, 129, 137, 13]	[85, 133, 141, 14]
20	one	[174, 176, 177, 1]	[1, 8, 14, 13, 16]	[130, 140, 139, 1]	[139, 153, 154, 1]
21	one	[122, 117, 125, 1]	[5, 7, 11, 15, 17]	[181, 155, 163, 1]	[170, 172, 178, 1]
22	one	[162, 153, 152, 16]	[159, 156, 154, 47]	[112, 8, 158, 165]	[83, 150, 163, 16]
23	one	[97, 107, 104, 13]	[49, 29, 25, 19, 2]	[160, 154, 151, 1]	[146, 152, 143, 1]

圖(五) 蒐集後的csv

(三) 研究結果與討論

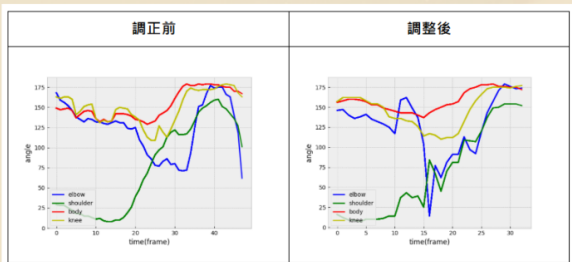
開始分析前先定義一下：球投出時=手肘（藍色）開始上升時，因為不管你的動作再怎麼奇怪，你都必須將你的小臂揮出，東西才可能丟的比較遠，你不妨試試看。

名稱	一段式(one motion)	二段式(two motion)
代表人物	Stephen Curry、Trae Young	Kobe Bryant、Ray Allen、林志傑
圖表		
圖表分析	發力順暢、甩鞭效應、投的遠	先跳再投、0.3s到0.5s滯空、出手點高、不易被蓋火鍋

圖(七) Stephen Curry 一段式投籃動作的關節角度變化

圖(八) 林志傑 二段式投籃動作的關節角度變化

回到我的目標投三分球，可以看到在調整前我的球投出時髖和膝已經打直，看起來跟林志傑一樣，但two motion需要靠著強大的核心和手臂力量來投籃，一般人如果想要更輕鬆投到三分可以往one motion的方式學，因為可以利用類似甩鞭效應來將力量從腳底往手最後在到球，讓力量在傳遞過程中有最小的損失。修改動作後，我的四條線能同時上升，也能有效運用到髖和膝蓋的力量。最終，我能輕鬆的投到了三分了！



(四) 結論應用及展望

從結果推知one motion和two motion有各自的優勢，於是產生了像Devin Booker的混和型投手，在三分線外用one motion而在三分內則用two motion，放大兩種各自的優勢。我希望未來可以以再新增

1. 自動辨識起跳、起球、投球階段的AI功能，分析時能更輕鬆判斷現在的動作大概對應到哪
2. 編輯影片開始結尾、放上色塊功能，而不用另外使用剪輯軟體。
3. 打包成exe，讓有興趣的人使用（可能電腦環境不乾淨，嘗試了很久都沒成功）

同時我現在也在做類似的事案，運用上述功能加上yolo追蹤籃球，除了分析關節角度，也分析球的起落速度的改變，一探絲滑運球秘密！